BASELINE CODES

1. **DCCRN [2]:** Deep Complex Convolution Recurrent Network for Phase-Aware Speech Enhancement

Non-Causal implementation code for DCCRN:

```
"""
Non-causal:
- Bidirectional LSTM at the bottleneck reads each sequence in forward
and
  backward directions.
- SAME-padded 2D convolutions use context on both sides of a time
frame.
"""
import tensorflow as tf
from tensorflow.keras import layers, Model


def conv_block(input_features, filters, name):
    conv_features_1 = layers.Conv2D(
        filters, 3, padding="same", use_bias=False,
        name=f"{name}_conv1"
    )(input_features)

    normalized_features_1 = layers.LayerNormalization(
        name=f"{name}_bn1"
    )(conv_features_1)

    activated_features_1 = layers.PReLU(
        shared_axes=[1, 2], name=f"{name}_prelu1"
    )(normalized_features_1)

    conv_features_2 = layers.Conv2D(
        filters, 3, padding="same", use_bias=False,
        name=f"{name}_conv2"
    )(activated_features_1)

    normalized_features_2 = layers.BatchNormalization(
        name=f"{name}_bn2"
    )(conv_features_2)

    output_features = layers.PReLU(
        shared_axes=[1, 2], name=f"{name}_prelu2"
    )(normalized_features_2)

    return output_features
```

```
def encoder_stage(input_features, filters, name):
    encoder_skip_features = conv_block(
        input_features, filters, name
    )

    downsampled_features = layers.Conv2D(
        filters * 2,
        kernel_size=3,
        strides=2,
        padding="same",
        activation="relu",
        name=f"{name}_downsample"
    )(encoder_skip_features)

    return encoder_skip_features, downsampled_features


def decoder_stage(input_features, skip_features, filters, name):
    upsampled_features = layers.Conv2DTranspose(
        filters,
        kernel_size=3,
        strides=2,
        padding="same",
        activation="relu",
        name=f"{name}_transpose_conv"
    )(input_features)

    skip_fused_features = layers.Concatenate(
        name=f"{name}_skip_concat"
    )([upsampled_features, skip_features])

    decoder_output_features = conv_block(
        skip_fused_features, filters, f"{name}_features"
    )

    return decoder_output_features


def build_dccrn_inspired_noncausal(
    input_shape=(128, 128, 1),
    base_filters=16,
    lstm_units=128
):
    noisy_spectrogram_input = layers.Input(
        shape=input_shape,
        name="noisy_spectrogram"
    )

    # Encoder stage 1
```

```
encoder1_skip, encoder1_downsampled = encoder_stage(
    noisy_spectrogram_input,
    base_filters,
    "encoder1"
)

# Encoder stage 2
encoder2_skip, encoder2_downsampled = encoder_stage(
    encoder1_downsampled,
    base_filters * 2,
    "encoder2"
)

# Encoder stage 3
encoder3_skip, encoder3_downsampled = encoder_stage(
    encoder2_downsampled,
    base_filters * 4,
    "encoder3"
)

# Encoder stage 4
encoder4_skip, encoder4_downsampled = encoder_stage(
    encoder3_downsampled,
    base_filters * 8,
    "encoder4"
)

# Bottleneck dimensions
bottleneck_height = int(encoder4_downsampled.shape[1])
bottleneck_width = int(encoder4_downsampled.shape[2])
bottleneck_channels = int(encoder4_downsampled.shape[3])

# Convert bottleneck features into a sequence
recurrent_input_sequence = layers.Reshape(
    (bottleneck_height, bottleneck_width * bottleneck_channels),
    name="bottleneck_sequence_reshape"
)(encoder4_downsampled)

# Bidirectional LSTM for past and future context
bidirectional_context_features = layers.Bidirectional(
    layers.LSTM(lstm_units, return_sequences=True),
    merge_mode="concat",
    name="bidirectional_lstm"
)(recurrent_input_sequence)

# Project recurrent features back to bottleneck feature dimensions
projected_context_features = layers.Dense(
    bottleneck_width * bottleneck_channels,
```

```
    activation="relu",
    name="bottleneck_projection"
)(bidirectional_context_features)

bottleneck_reshaped_features = layers.Reshape(
    (bottleneck_height, bottleneck_width, bottleneck_channels),
    name="bottleneck_feature_reshape"
)(projected_context_features)

bottleneck_output_features = conv_block(
    bottleneck_reshaped_features,
    base_filters * 16,
    "bottleneck_conv"
)

# Decoder stage 4
decoder4_output = decoder_stage(
    bottleneck_output_features,
    encoder4_skip,
    base_filters * 8,
    "decoder4"
)

# Decoder stage 3
decoder3_output = decoder_stage(
    decoder4_output,
    encoder3_skip,
    base_filters * 4,
    "decoder3"
)

# Decoder stage 2
decoder2_output = decoder_stage(
    decoder3_output,
    encoder2_skip,
    base_filters * 2,
    "decoder2"
)

# Decoder stage 1
decoder1_output = decoder_stage(
    decoder2_output,
    encoder1_skip,
    base_filters,
    "decoder1"
)

# Estimate enhanced magnitude spectrogram
```

```
    enhanced_spectrogram_output = layers.Conv2D(
        1,
        kernel_size=1,
        padding="same",
        activation="softplus",
        name="enhanced_spectrogram"
    )(decoder1_output)

    speech_enhancement_model = Model(
        inputs=noisy_spectrogram_input,
        outputs=enhanced_spectrogram_output,
        name="DCCRN_Inspired_NonCausal_TF"
    )

    return speech_enhancement_model


def spectral_loss(clean_spectrogram, predicted_spectrogram):
    magnitude_absolute_error = tf.reduce_mean(
        tf.abs(clean_spectrogram - predicted_spectrogram)
    )

    magnitude_squared_error = tf.reduce_mean(
        tf.square(clean_spectrogram - predicted_spectrogram)
    )

    combined_spectral_loss = (
        magnitude_absolute_error
        + 0.5 * magnitude_squared_error
    )

    return combined_spectral_loss


if __name__ == "__main__":
    speech_enhancement_model = build_dccrn_inspired_noncausal(
        input_shape=(128, 128, 1),
        base_filters=16,
        lstm_units=128
    )

    speech_enhancement_model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=1e-3
        ),
        loss=spectral_loss,
        metrics=[
            tf.keras.metrics.MeanAbsoluteError(name="mae")
        ]
```

```
    )

    speech_enhancement_model.summary()
```

2. MCGN [1]: A Multi-Scale Feature Recalibration Network for End-to-End Single Channel Speech Enhancement [18]

Non-Causal implementation code for MCGN:

```
import tensorflow as tf
from tensorflow.keras import layers, Model


def conv_bn_relu(input_features, filters, kernel_size,
                 strides=(1, 1), transpose=False, name="conv"):

    convolution_layer = (
        layers.Conv2DTranspose if transpose else layers.Conv2D
    )

    convolution_features = convolution_layer(
        filters,
        kernel_size,
        strides=strides,
        padding="same",
        use_bias=False,
        name=f"{name}_conv"
    )(input_features)

    normalized_features = layers.LayerNormalization(
        name=f"{name}_bn"
    )(convolution_features)

    activated_features = layers.ReLU(
        name=f"{name}_relu"
    )(normalized_features)

    return activated_features


def mcfr_block(input_features, filters, name="mcfr"):
    branch_filters = max(filters // 5, 4)
    multiscale_branches = []

    # Five parallel multi-scale convolution branches
    for scale_index, kernel_size in enumerate(
        (1, 3, 5, 7, 9), start=1
    ):
```

```
        scale_features = conv_bn_relu(
            input_features,
            branch_filters,
            (kernel_size, kernel_size),
            name=f"{name}_scale{scale_index}"
        )
        multiscale_branches.append(scale_features)

    concatenated_multiscale_features = layers.Concatenate(
        name=f"{name}_multiscale_concat"
    )(multiscale_branches)

    projected_multiscale_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        use_bias=False,
        name=f"{name}_feature_projection"
    )(concatenated_multiscale_features)

    # Channel-attention gate
    global_channel_descriptor = layers.GlobalAveragePooling2D(
        name=f"{name}_gap"
    )(projected_multiscale_features)

    reduced_channel_features = layers.Dense(
        max(filters // 4, 4),
        activation="relu",
        name=f"{name}_fc1"
    )(global_channel_descriptor)

    channel_attention_weights = layers.Dense(
        filters,
        activation="sigmoid",
        name=f"{name}_fc2"
    )(reduced_channel_features)

    reshaped_attention_weights = layers.Reshape(
        (1, 1, filters),
        name=f"{name}_gate_reshape"
    )(channel_attention_weights)

    attention_weighted_features = layers.Multiply(
        name=f"{name}_multiply"
    )([
        projected_multiscale_features,
        reshaped_attention_weights
    ])
```

```
    residual_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        name=f"{name}_shortcut"
    )(input_features)

    fused_residual_features = layers.Add(
        name=f"{name}_residual_sum"
    )([
        residual_features,
        attention_weighted_features
    ])

    output_features = layers.ReLU(
        name=f"{name}_output_relu"
    )(fused_residual_features)

    return output_features


def encoder_stage(input_features, filters, name):
    multiscale_features = mcfr_block(
        input_features,
        filters,
        name=f"{name}_mcfr"
    )

    encoder_skip_features = multiscale_features

    downsampled_features = layers.Conv2D(
        filters * 2,
        3,
        strides=2,
        padding="same",
        use_bias=False,
        name=f"{name}_downsample"
    )(multiscale_features)

    normalized_downsampled_features = layers.LayerNormalization(
        name=f"{name}_down_bn"
    )(downsampled_features)

    activated_downsampled_features = layers.ReLU(
        name=f"{name}_down_relu"
    )(normalized_downsampled_features)

    return activated_downsampled_features, encoder_skip_features
```

```
def decoder_stage(input_features, skip_features, filters, name):
    upsampled_features = layers.Conv2DTranspose(
        filters,
        4,
        strides=2,
        padding="same",
        use_bias=False,
        name=f"{name}_upsample"
    )(input_features)

    normalized_upsampled_features = layers.LayerNormalization(
        name=f"{name}_up_bn"
    )(upsampled_features)

    activated_upsampled_features = layers.ReLU(
        name=f"{name}_up_relu"
    )(normalized_upsampled_features)

    skip_fused_features = layers.Concatenate(
        name=f"{name}_skip_concat"
    )([
        activated_upsampled_features,
        skip_features
    ])

    decoder_output_features = mcfr_block(
        skip_fused_features,
        filters,
        name=f"{name}_mcfr"
    )

    return decoder_output_features


def build_mcgn(
    input_shape=(128, 128, 1),
    base_filters=16,
    gru_units=128
):
    noisy_spectrogram_input = layers.Input(
        shape=input_shape,
        name="noisy_spectrogram"
    )

    # Initial feature extraction
    initial_spectrogram_features = conv_bn_relu(
        noisy_spectrogram_input,
```

```
        base_filters,
        (3, 3),
        name="input_projection"
    )

    # Encoder: 128 -> 64 -> 32 -> 16 -> 8
    encoder1_downsampled, encoder1_skip = encoder_stage(
        initial_spectrogram_features,
        base_filters,
        "encoder1"
    )

    encoder2_downsampled, encoder2_skip = encoder_stage(
        encoder1_downsampled,
        base_filters * 2,
        "encoder2"
    )

    encoder3_downsampled, encoder3_skip = encoder_stage(
        encoder2_downsampled,
        base_filters * 4,
        "encoder3"
    )

    encoder4_downsampled, encoder4_skip = encoder_stage(
        encoder3_downsampled,
        base_filters * 8,
        "encoder4"
    )

    # Multi-scale convolutional bottleneck
    bottleneck_multiscale_features = mcfr_block(
        encoder4_downsampled,
        base_filters * 16,
        name="bottleneck_mcfr"
    )

    # Connection layers: reshape -> FC reduction -> BiGRU -> FC
expansion
    bottleneck_height = int(bottleneck_multiscale_features.shape[1])
    bottleneck_width = int(bottleneck_multiscale_features.shape[2])
    bottleneck_channels = int(bottleneck_multiscale_features.shape[3])

    recurrent_input_sequence = layers.Reshape(
        (
            bottleneck_height,
            bottleneck_width * bottleneck_channels
        ),
```

```
    name="connection_reshape"
)(bottleneck_multiscale_features)

reduced_recurrent_features = layers.Dense(
    1024,
    activation="relu",
    name="connection_fc_reduce"
)(recurrent_input_sequence)

bidirectional_gru_features = layers.Bidirectional(
    layers.GRU(gru_units, return_sequences=True),
    name="connection_bigru"
)(reduced_recurrent_features)

expanded_recurrent_features = layers.Dense(
    bottleneck_width * bottleneck_channels,
    activation="relu",
    name="connection_fc_expand"
)(bidirectional_gru_features)

restored_bottleneck_features = layers.Reshape(
    (
        bottleneck_height,
        bottleneck_width,
        bottleneck_channels
    ),
    name="connection_restore"
)(expanded_recurrent_features)

# Decoder with skip connections
decoder4_features = decoder_stage(
    restored_bottleneck_features,
    encoder4_skip,
    base_filters * 8,
    "decoder4"
)

decoder3_features = decoder_stage(
    decoder4_features,
    encoder3_skip,
    base_filters * 4,
    "decoder3"
)

decoder2_features = decoder_stage(
    decoder3_features,
    encoder2_skip,
    base_filters * 2,
```

```
        "decoder2"
    )

    decoder1_features = decoder_stage(
        decoder2_features,
        encoder1_skip,
        base_filters,
        "decoder1"
    )

    # Enhanced spectrogram estimation
    decoder_spectrogram_estimate = layers.Conv2D(
        1,
        1,
        padding="same",
        activation="linear",
        name="enhanced_spectrogram"
    )(decoder1_features)

    input_residual_estimate = layers.Conv2D(
        1,
        1,
        padding="same",
        activation="linear",
        name="input_residual_projection"
    )(noisy_spectrogram_input)

    enhanced_spectrogram_output = layers.Add(
        name="enhanced_output"
    )([
        decoder_spectrogram_estimate,
        input_residual_estimate
    ])

    mcgn_model = Model(
        noisy_spectrogram_input,
        enhanced_spectrogram_output,
        name="MCGN_inspired_2D"
    )

    return mcgn_model


if __name__ == "__main__":
    speech_enhancement_model = build_mcgn(
        input_shape=(128, 128, 1),
        base_filters=16,
        gru_units=128
    )
```

```
speech_enhancement_model.summary()
```

3. MASENet [12]: A Convolutional Network With Multi-Scale and Attention Mechanisms for End-to-End Single-Channel Speech Enhancement

Non-Causal implementation code for MASENet:

```
import tensorflow as tf
from tensorflow.keras import layers, Model


def conv_norm_elu(input_features, filters, kernel_size, name,
                  dilation_rate=1):
    """Convolution -> Layer Normalization -> ELU activation."""
    convolution_features = layers.Conv2D(
        filters,
        kernel_size,
        padding="same",
        dilation_rate=dilation_rate,
        use_bias=False,
        name=f"{name}_conv"
    )(input_features)

    normalized_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_ln"
    )(convolution_features)

    activated_features = layers.ELU(
        name=f"{name}_elu"
    )(normalized_features)

    return activated_features


def attention_block(input_features, name, reduction=8):
    """CBAM-inspired channel and spatial attention."""
    channel_count = int(input_features.shape[-1])
    reduced_channel_count = max(channel_count // reduction, 4)

    # Channel attention: average and maximum pooled descriptors.
    average_channel_descriptor = layers.GlobalAveragePooling2D(
        keepdims=True,
        name=f"{name}_avgpool"
    )(input_features)
```

```
maximum_channel_descriptor = layers.GlobalMaxPooling2D(
    keepdims=True,
    name=f"{name}_maxpool"
)(input_features)

shared_channel_reduction = layers.Conv2D(
    reduced_channel_count,
    1,
    activation="relu",
    padding="same",
    name=f"{name}_shared_reduce"
)

shared_channel_expansion = layers.Conv2D(
    channel_count,
    1,
    padding="same",
    name=f"{name}_shared_expand"
)

average_channel_features = shared_channel_expansion(
    shared_channel_reduction(average_channel_descriptor)
)

maximum_channel_features = shared_channel_expansion(
    shared_channel_reduction(maximum_channel_descriptor)
)

combined_channel_descriptors = layers.Add(
    name=f"{name}_channel_add"
)([
    average_channel_features,
    maximum_channel_features
])

channel_attention_weights = layers.Activation(
    "sigmoid",
    name=f"{name}_channel_sigmoid"
)(combined_channel_descriptors)

channel_refined_features = layers.Multiply(
    name=f"{name}_channel_refine"
)([
    input_features,
    channel_attention_weights
])
```

```
    # Spatial attention: channel-wise average and maximum descriptors.
    average_spatial_descriptor = layers.Lambda(
        lambda feature_map: tf.reduce_mean(
            feature_map, axis=-1, keepdims=True
        ),
        name=f"{name}_spatial_avg"
    )(channel_refined_features)

    maximum_spatial_descriptor = layers.Lambda(
        lambda feature_map: tf.reduce_max(
            feature_map, axis=-1, keepdims=True
        ),
        name=f"{name}_spatial_max"
    )(channel_refined_features)

    combined_spatial_descriptors = layers.Concatenate(
        name=f"{name}_spatial_concat"
    )([
        average_spatial_descriptor,
        maximum_spatial_descriptor
    ])

    spatial_attention_weights = layers.Conv2D(
        1,
        7,
        padding="same",
        activation="sigmoid",
        name=f"{name}_spatial_gate"
    )(combined_spatial_descriptors)

    spatial_refined_features = layers.Multiply(
        name=f"{name}_spatial_refine"
    )([
        channel_refined_features,
        spatial_attention_weights
    ])

    return spatial_refined_features


def multi_scale_speech_encoder(input_features, filters=32,
                               name="speech_encoder"):
    """Parallel short-, middle-, long-context and pooling branches."""

    short_context_features = conv_norm_elu(
        input_features, filters, (1, 3),
        f"{name}_short"
    )
```

```
    middle_context_features = conv_norm_elu(
        input_features, filters, (1, 5),
        f"{name}_middle"
    )

    long_context_features = conv_norm_elu(
        input_features, filters, (1, 7),
        f"{name}_long"
    )

    pooled_context_features = layers.AveragePooling2D(
        pool_size=(1, 3),
        strides=(1, 1),
        padding="same",
        name=f"{name}_avgpool"
    )(input_features)

    projected_pooling_features = conv_norm_elu(
        pooled_context_features, filters, (1, 1),
        f"{name}_pool_proj"
    )

    concatenated_multiscale_features = layers.Concatenate(
        name=f"{name}_multiscale_concat"
    )([
        short_context_features,
        middle_context_features,
        long_context_features,
        projected_pooling_features
    ])

    fused_multiscale_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        name=f"{name}_fusion"
    )(concatenated_multiscale_features)

    attention_refined_multiscale_features = attention_block(
        fused_multiscale_features,
        f"{name}_attention"
    )

    return attention_refined_multiscale_features


def frequency_dilated_module(input_features, filters=32,
                             name="frequency_dilated"):
    """Parallel dilated frequency convolutions with feature fusion."""
```

```
    dilation_configurations = [
        (filters, 1),
        (32, 2),
        (16, 4),
        (16, 8)
    ]

    frequency_context_branches = []

    for branch_index, (branch_filters, frequency_dilation) in
enumerate(
        dilation_configurations, start=1
    ):
        frequency_branch_features = conv_norm_elu(
            input_features,
            branch_filters,
            (3, 1),
            f"{name}_branch{branch_index}",
            dilation_rate=(frequency_dilation, 1)
        )

        frequency_context_branches.append(frequency_branch_features)

    concatenated_frequency_features = layers.Concatenate(
        name=f"{name}_dense_concat"
    )(frequency_context_branches)

    projected_frequency_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        name=f"{name}_projection"
    )(concatenated_frequency_features)

    frequency_residual_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        name=f"{name}_res_proj"
    )(frequency_context_branches[0])

    fused_frequency_features = layers.Add(
        name=f"{name}_residual"
    )([
        frequency_residual_features,
        projected_frequency_features
    ])
```

```
    return fused_frequency_features


def temporal_convolutional_attention(input_features, filters=32,
                                     name="tca", stages=3):
    """Dilated temporal convolutions with global temporal-channel
gating."""

    temporal_residual_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        name=f"{name}_residual_proj"
    )(input_features)

    temporal_context_features = conv_norm_elu(
        input_features,
        filters,
        (1, 1),
        f"{name}_input"
    )

    for stage_index in range(stages):
        temporal_dilation_rate = 2 ** stage_index

        temporal_context_features = conv_norm_elu(
            temporal_context_features,
            filters,
            (1, 3),
            f"{name}_temporal{stage_index + 1}",
            dilation_rate=(1, temporal_dilation_rate)
        )

    global_temporal_descriptor = layers.GlobalAveragePooling2D(
        keepdims=True,
        name=f"{name}_global_avg"
    )(temporal_context_features)

    reduced_temporal_channel_features = layers.Conv2D(
        filters,
        1,
        activation="relu",
        padding="same",
        name=f"{name}_gate_reduce"
    )(global_temporal_descriptor)

    temporal_attention_weights = layers.Conv2D(
        filters,
```

```
            1,
            activation="sigmoid",
            padding="same",
            name=f"{name}_gate_sigmoid"
        )(reduced_temporal_channel_features)

        attention_refined_temporal_features = layers.Multiply(
            name=f"{name}_apply_gate"
        )([
            temporal_context_features,
            temporal_attention_weights
        ])

        projected_temporal_features = layers.Conv2D(
            filters,
            1,
            padding="same",
            name=f"{name}_output_proj"
        )(attention_refined_temporal_features)

        fused_temporal_features = layers.Add(
            name=f"{name}_residual_add"
        )([
            temporal_residual_features,
            projected_temporal_features
        ])

        return fused_temporal_features


    def post_processing_module(input_features, name="postprocess"):
        """Two convolution-normalization-activation refinement layers."""

        input_channel_count = int(input_features.shape[-1])

        first_refinement_features = conv_norm_elu(
            input_features,
            input_channel_count,
            (1, 1),
            f"{name}_conv1"
        )

        second_refinement_features = conv_norm_elu(
            first_refinement_features,
            input_channel_count,
            (1, 1),
            f"{name}_conv2"
        )
```

```
    return second_refinement_features


def build_masenet_inspired(input_shape=(128, 128, 1), filters=32):
    """Build a non-causal, MASENet-inspired magnitude enhancement
model."""

    noisy_magnitude_spectrogram = layers.Input(
        shape=input_shape,
        name="noisy_magnitude_spectrogram"
    )

    # Module 1: multiscale speech feature extraction and attention.
    multiscale_encoder_features = multi_scale_speech_encoder(
        noisy_magnitude_spectrogram,
        filters,
        "multi_scale_encoder"
    )

    # Module 2: frequency-dilated feature extraction.
    frequency_dilated_features = frequency_dilated_module(
        multiscale_encoder_features,
        filters,
        "frequency_dilated_module"
    )

    # Module 3: temporal convolutional attention.
    temporal_attention_features = temporal_convolutional_attention(
        frequency_dilated_features,
        filters,
        "temporal_attention_module"
    )

    # Module 4: post-processing refinement.
    post_processed_features = post_processing_module(
        temporal_attention_features,
        "post_processing_module"
    )

    # Module 5: magnitude-mask estimation and residual refinement.
    estimated_magnitude_mask = layers.Conv2D(
        1,
        1,
        padding="same",
        activation="sigmoid",
        name="magnitude_mask"
    )(post_processed_features)

    mask_refined_magnitude = layers.Multiply(
```

```
        name="apply_magnitude_mask"
    )([
        noisy_magnitude_spectrogram,
        estimated_magnitude_mask
    ])

    residual_spectrogram_estimate = layers.Conv2D(
        1,
        3,
        padding="same",
        activation="linear",
        name="residual_spectrogram"
    )(post_processed_features)

    combined_magnitude_estimate = layers.Add(
        name="masked_plus_residual"
    )([
        mask_refined_magnitude,
        residual_spectrogram_estimate
    ])

    enhanced_magnitude_spectrogram = layers.Activation(
        "softplus",
        name="enhanced_magnitude"
    )(combined_magnitude_estimate)

    masenet_inspired_model = Model(
        inputs=noisy_magnitude_spectrogram,
        outputs=enhanced_magnitude_spectrogram,
        name="MASENet_Inspired_NonCausal"
    )

    return masenet_inspired_model


def magnitude_loss(y_true, y_pred):
    """Combined L1 and weighted MSE magnitude loss."""

    absolute_magnitude_error = tf.reduce_mean(
        tf.abs(y_true - y_pred)
    )

    squared_magnitude_error = tf.reduce_mean(
        tf.square(y_true - y_pred)
    )

    combined_magnitude_loss = (
        absolute_magnitude_error
        + 0.5 * squared_magnitude_error
```

```
    )

    return combined_magnitude_loss


if __name__ == "__main__":
    masenet_model = build_masenet_inspired(
        input_shape=(128, 128, 1),
        filters=32
    )

    masenet_model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss=magnitude_loss,
        metrics=[
            tf.keras.metrics.MeanAbsoluteError(name="mae")
        ]
    )

    masenet_model.summary()
```

4. SADNUNet [13]: A Nested U-Net With Self-Attention and Dense Connectivity for Monaural Speech Enhancement

Non-Causal implementation code for SADNUNet:

```
import tensorflow as tf
from tensorflow.keras import layers, Model


# ------------------------------------------------------------
# Convolution, normalization, and activation block
# ------------------------------------------------------------
def conv_block(input_features, filters, name):
    convolution_features = layers.Conv2D(
        filters,
        kernel_size=3,
        strides=2,
        padding="same",
        use_bias=False,
        name=f"{name}_conv"
    )(input_features)

    normalized_features = layers.LayerNormalization(
        name=f"{name}_bn"
    )(convolution_features)

    activated_features = layers.PReLU(
```

```
        shared_axes=[1, 2],
        name=f"{name}_prelu"
    )(normalized_features)

    return activated_features


# ------------------------------------------------------------
# Bilinear upsampling, convolution, normalization, and activation
# ------------------------------------------------------------
def upsample_block(input_features, filters, name):
    upsampled_features = layers.UpSampling2D(
        size=(2, 2),
        interpolation="bilinear",
        name=f"{name}_upsample"
    )(input_features)

    convolution_features = layers.Conv2D(
        filters,
        kernel_size=3,
        padding="same",
        use_bias=False,
        name=f"{name}_conv"
    )(upsampled_features)

    normalized_features = layers.LayerNormalization(
        name=f"{name}_bn"
    )(convolution_features)

    activated_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_prelu"
    )(normalized_features)

    return activated_features


# ------------------------------------------------------------
# Resize feature map to an explicit spatial resolution
# ------------------------------------------------------------
def resize_to(input_features, height, width, name):
    resized_features = layers.Resizing(
        height=height,
        width=width,
        interpolation="bilinear",
        name=name
    )(input_features)

    return resized_features
```

```python
# ------------------------------------------------------------
# Dense feature fusion for nested skip connections
# ------------------------------------------------------------
def dense_fusion(input_feature_maps, filters, name):
    target_feature_shape = tf.keras.backend.int_shape(
        input_feature_maps[0]
    )

    target_height = target_feature_shape[1]
    target_width = target_feature_shape[2]

    spatially_aligned_features = [input_feature_maps[0]]

    for feature_index, feature_map in enumerate(
        input_feature_maps[1:], start=1
    ):
        feature_shape = tf.keras.backend.int_shape(feature_map)

        feature_height = feature_shape[1]
        feature_width = feature_shape[2]

        if (
            feature_height != target_height
            or feature_width != target_width
        ):
            feature_map = resize_to(
                feature_map,
                target_height,
                target_width,
                name=f"{name}_resize{feature_index}"
            )

        spatially_aligned_features.append(feature_map)

    concatenated_skip_features = layers.Concatenate(
        name=f"{name}_concat"
    )(spatially_aligned_features)

    fused_convolution_features = layers.Conv2D(
        filters,
        kernel_size=3,
        padding="same",
        use_bias=False,
        name=f"{name}_conv"
    )(concatenated_skip_features)

    normalized_fusion_features = layers.LayerNormalization(
```

```
            name=f"{name}_bn"
        )(fused_convolution_features)

        activated_fusion_features = layers.PReLU(
            shared_axes=[1, 2],
            name=f"{name}_prelu"
        )(normalized_fusion_features)

        return activated_fusion_features


# ---------------------------------------------------------------
# Spatial self-attention with a trainable residual scale
# ---------------------------------------------------------------
class SpatialSelfAttention(layers.Layer):

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

    def build(self, input_shape):
        input_channel_count = int(input_shape[-1])
        attention_channel_count = max(input_channel_count // 8, 1)

        self.query_projection = layers.Conv2D(
            attention_channel_count,
            1,
            padding="same",
            name="query"
        )

        self.key_projection = layers.Conv2D(
            attention_channel_count,
            1,
            padding="same",
            name="key"
        )

        self.value_projection = layers.Conv2D(
            input_channel_count,
            1,
            padding="same",
            name="value"
        )

        self.attention_residual_scale = self.add_weight(
            name="gamma",
            shape=(),
            initializer="zeros",
            trainable=True
```

```
        )

        super().build(input_shape)

    def call(self, input_features):
        dynamic_shape = tf.shape(input_features)

        batch_size = dynamic_shape[0]
        feature_height = dynamic_shape[1]
        feature_width = dynamic_shape[2]

        input_channel_count = input_features.shape[-1]

        # Generate query, key, and value feature maps.
        query_features = self.query_projection(input_features)
        key_features = self.key_projection(input_features)
        value_features = self.value_projection(input_features)

        # Flatten spatial dimensions into a sequence of positions.
        flattened_query_features = tf.reshape(
            query_features,
            [batch_size, feature_height * feature_width, -1]
        )

        flattened_key_features = tf.reshape(
            key_features,
            [batch_size, feature_height * feature_width, -1]
        )

        flattened_value_features = tf.reshape(
            value_features,
            [
                batch_size,
                feature_height * feature_width,
                input_channel_count
            ]
        )

        # Calculate scaled spatial attention.
        query_key_similarity = tf.matmul(
            flattened_query_features,
            flattened_key_features,
            transpose_b=True
        )

        attention_scale_factor = tf.sqrt(
            tf.cast(
                tf.shape(flattened_query_features)[-1],
```

```
                query_key_similarity.dtype
            )
        )

        spatial_attention_weights = tf.nn.softmax(
            query_key_similarity / attention_scale_factor,
            axis=-1
        )

        # Aggregate value features using attention weights.
        attended_spatial_features = tf.matmul(
            spatial_attention_weights,
            flattened_value_features
        )

        restored_attention_features = tf.reshape(
            attended_spatial_features,
            [
                batch_size,
                feature_height,
                feature_width,
                input_channel_count
            ]
        )

        attention_refined_features = (
            input_features
            + self.attention_residual_scale
            * restored_attention_features
        )

        return attention_refined_features


# ------------------------------------------------------------
# SADNUNet-inspired spectrogram enhancement model
# Input/output shape: (128, 128, 1)
# ------------------------------------------------------------
def build_sadnunet_inspired(
    input_shape=(128, 128, 1),
    base_filters=16
):
    noisy_spectrogram_input = layers.Input(
        shape=input_shape,
        name="noisy_spectrogram"
    )

    # ------------------------------------------------------
    # Encoder: 128x128 -> 64x64 -> 32x32 -> 16x16 -> 8x8
```

```
    # ----------------------------------------------------------
    encoder1_features = conv_block(
        noisy_spectrogram_input,
        base_filters,
        "enc1"
    )

    encoder2_features = conv_block(
        encoder1_features,
        base_filters * 2,
        "enc2"
    )

    encoder3_features = conv_block(
        encoder2_features,
        base_filters * 4,
        "enc3"
    )

    encoder4_features = conv_block(
        encoder3_features,
        base_filters * 8,
        "enc4"
    )

    # ----------------------------------------------------------
    # Bottleneck: spatial self-attention at 8x8
    # ----------------------------------------------------------
    bottleneck_attention_features = SpatialSelfAttention(
        name="bottleneck_attention"
    )(encoder4_features)

    # ----------------------------------------------------------
    # Nested decoder pathway
    # ----------------------------------------------------------

    # Nested node (0,1): fuse 64x64 encoder and upsampled 32x32
features.
    nested01_upsampled_features = upsample_block(
        encoder2_features,
        base_filters,
        "up01"
    )

    nested01_fused_features = dense_fusion(
        [
            encoder1_features,
            nested01_upsampled_features
```

```
        ],
        base_filters,
        "nest01"
    )

    # Nested node (1,1): fuse 32x32 encoder and upsampled 16x16
features.
    nested11_upsampled_features = upsample_block(
        encoder3_features,
        base_filters * 2,
        "up11"
    )

    nested11_fused_features = dense_fusion(
        [
            encoder2_features,
            nested11_upsampled_features
        ],
        base_filters * 2,
        "nest11"
    )

    # Nested node (0,2): combine multiple 64x64 feature pathways.
    nested02_upsampled_features = upsample_block(
        nested11_fused_features,
        base_filters,
        "up02"
    )

    nested02_fused_features = dense_fusion(
        [
            encoder1_features,
            nested01_fused_features,
            nested02_upsampled_features
        ],
        base_filters,
        "nest02"
    )

    # ------------------------------------------------------------
    # Attention pathway: 8x8 -> 16x16 -> 32x32 -> 64x64
    # ------------------------------------------------------------
    attention_stage1_upsampled_features = upsample_block(
        bottleneck_attention_features,
        base_filters * 4,
        "attn_up1"
    )
```

```
attention_stage1_features = resize_to(
    attention_stage1_upsampled_features,
    16,
    16,
    "attn_resize1"
)

attention_stage2_upsampled_features = upsample_block(
    attention_stage1_features,
    base_filters * 2,
    "attn_up2"
)

attention_stage2_features = resize_to(
    attention_stage2_upsampled_features,
    32,
    32,
    "attn_resize2"
)

attention_stage3_upsampled_features = upsample_block(
    attention_stage2_features,
    base_filters,
    "attn_up3"
)

attention_stage3_features = resize_to(
    attention_stage3_upsampled_features,
    64,
    64,
    "attn_resize3"
)

# ------------------------------------------------------------
# Final fusion at 64x64
# ------------------------------------------------------------
fused_decoder_attention_features = layers.Concatenate(
    name="final_fusion"
)([
    nested02_fused_features,
    attention_stage3_features
])

final_refinement_features = layers.Conv2D(
    base_filters,
    kernel_size=3,
    padding="same",
    activation="relu",
```

```
        name="final_conv"
    )(fused_decoder_attention_features)

    # ------------------------------------------------------------
    # Output reconstruction: 64x64 -> 128x128
    # ------------------------------------------------------------
    full_resolution_features = layers.UpSampling2D(
        size=(2, 2),
        interpolation="bilinear",
        name="output_upsample"
    )(final_refinement_features)

    enhanced_spectrogram_output = layers.Conv2D(
        1,
        kernel_size=1,
        padding="same",
        activation="relu",
        name="enhanced_spectrogram"
    )(full_resolution_features)

    sadnunet_inspired_model = Model(
        inputs=noisy_spectrogram_input,
        outputs=enhanced_spectrogram_output,
        name="SADNUNet_Inspired"
    )

    sadnunet_inspired_model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=1e-3
        ),
        loss="mae",
        metrics=["mse"]
    )

    return sadnunet_inspired_model


# ------------------------------------------------------------
# Build model, verify output shape, and report parameters
# ------------------------------------------------------------
if __name__ == "__main__":
    sadnunet_model = build_sadnunet_inspired(
        input_shape=(128, 128, 1),
        base_filters=16
    )

    sadnunet_model.summary()
```

5. MANNER [14]: Multi-View Attention Network for Noise Erasure

Non-Causal implementation code for MANNERNet

```
import tensorflow as tf
from tensorflow.keras import layers, Model


# ------------------------------------------------------------
# Convolution, normalization, and Swish activation
# ------------------------------------------------------------
def conv2d_block(input_features, filters, kernel_size=3,
                 strides=1, name="conv"):
    """Non-causal 2D convolution block using SAME padding."""

    convolution_features = layers.Conv2D(
        filters,
        kernel_size=kernel_size,
        strides=strides,
        padding="same",
        use_bias=False,
        name=f"{name}_conv"
    )(input_features)

    normalized_features = layers.LayerNormalization(
        name=f"{name}_bn"
    )(convolution_features)

    activated_features = layers.Activation(
        "swish",
        name=f"{name}_swish"
    )(normalized_features)

    return activated_features


# ------------------------------------------------------------
# Residual Conformer-inspired feature extraction block
# ------------------------------------------------------------
def rescon_block(input_features, filters, growth_rate=2,
                 name="rescon"):
    """
    Pointwise expansion -> GLU -> depthwise convolution ->
    Swish -> pointwise projection -> residual connection.
    """

    residual_shortcut_features = input_features

    # Pointwise channel expansion.
```

```
expanded_channel_count = filters * growth_rate * 2

expanded_convolution_features = layers.Conv2D(
    expanded_channel_count,
    1,
    padding="same",
    use_bias=False,
    name=f"{name}_pointwise_expand"
)(input_features)

normalized_expansion_features = layers.LayerNormalization(
    name=f"{name}_expand_bn"
)(expanded_convolution_features)

# Split features into value and gate components.
value_features, gate_features = layers.Lambda(
    lambda feature_map: tf.split(
        feature_map,
        num_or_size_splits=2,
        axis=-1
    ),
    name=f"{name}_glu_split"
)(normalized_expansion_features)

sigmoid_gate_features = layers.Activation(
    "sigmoid",
    name=f"{name}_glu_sigmoid"
)(gate_features)

gated_expansion_features = layers.Multiply(
    name=f"{name}_glu"
)([
    value_features,
    sigmoid_gate_features
])

# Depthwise spatial convolution.
depthwise_convolution_features = layers.DepthwiseConv2D(
    kernel_size=3,
    padding="same",
    use_bias=False,
    name=f"{name}_depthwise"
)(gated_expansion_features)

normalized_depthwise_features = layers.LayerNormalization(
    name=f"{name}_depthwise_bn"
)(depthwise_convolution_features)
```

```
activated_depthwise_features = layers.Activation(
    "swish",
    name=f"{name}_depthwise_swish"
)(normalized_depthwise_features)

# Pointwise projection to the requested channel count.
projected_convolution_features = layers.Conv2D(
    filters,
    1,
    padding="same",
    use_bias=False,
    name=f"{name}_pointwise_project"
)(activated_depthwise_features)

normalized_projection_features = layers.LayerNormalization(
    name=f"{name}_project_bn"
)(projected_convolution_features)

# Match residual channel dimensions when required.
if (
    tf.keras.backend.int_shape(residual_shortcut_features)[-1]
    != filters
):
    residual_shortcut_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        use_bias=False,
        name=f"{name}_shortcut"
    )(residual_shortcut_features)

    residual_shortcut_features = layers.BatchNormalization(
        name=f"{name}_shortcut_bn"
    )(residual_shortcut_features)

residual_fused_features = layers.Add(
    name=f"{name}_residual_add"
)([
    residual_shortcut_features,
    normalized_projection_features
])

output_features = layers.Activation(
    "relu",
    name=f"{name}_out_relu"
)(residual_fused_features)

return output_features
```

```python
# ------------------------------------------------------------
# Channel attention
# ------------------------------------------------------------
def channel_attention(input_features,
                      name="channel_attention",
                      reduction=4):
    """Channel attention using global average pooling."""

    channel_count = tf.keras.backend.int_shape(
        input_features
    )[-1]

    reduced_channel_count = max(
        channel_count // reduction,
        4
    )

    global_channel_descriptor = layers.GlobalAveragePooling2D(
        keepdims=True,
        name=f"{name}_gap"
    )(input_features)

    reduced_channel_features = layers.Conv2D(
        reduced_channel_count,
        1,
        activation="relu",
        name=f"{name}_reduce"
    )(global_channel_descriptor)

    channel_attention_weights = layers.Conv2D(
        channel_count,
        1,
        activation="sigmoid",
        name=f"{name}_expand"
    )(reduced_channel_features)

    channel_refined_features = layers.Multiply(
        name=f"{name}_apply"
    )([
        input_features,
        channel_attention_weights
    ])

    return channel_refined_features


# ------------------------------------------------------------
```

```
# Global attention view
# ------------------------------------------------------------
def global_attention_view(input_features,
                          name="global_attention",
                          max_side=8):
    """
    Global unmasked multi-head attention on a reduced spatial grid.
    """

    channel_count = tf.keras.backend.int_shape(
        input_features
    )[-1]

    feature_height = tf.keras.backend.int_shape(
        input_features
    )[1]

    feature_width = tf.keras.backend.int_shape(
        input_features
    )[2]

    pooled_height = min(feature_height, max_side)
    pooled_width = min(feature_width, max_side)

    # Reduce spatial resolution to control attention memory.
    pooled_spatial_features = layers.Resizing(
        pooled_height,
        pooled_width,
        interpolation="bilinear",
        name=f"{name}_pool"
    )(input_features)

    # Convert the spatial grid into a token sequence.
    spatial_attention_tokens = layers.Reshape(
        (pooled_height * pooled_width, channel_count),
        name=f"{name}_tokens"
    )(pooled_spatial_features)

    attention_head_count = (
        4 if channel_count >= 16 else 1
    )

    attention_key_dimension = max(
        channel_count // attention_head_count,
        4
    )

    # Unmasked self-attention over all tokens.
```

```
    globally_attended_tokens = layers.MultiHeadAttention(
        num_heads=attention_head_count,
        key_dim=attention_key_dimension,
        name=f"{name}_mha"
    )(
        spatial_attention_tokens,
        spatial_attention_tokens,
        use_causal_mask=False
    )

    attention_residual_tokens = layers.Add(
        name=f"{name}_token_residual"
    )([
        spatial_attention_tokens,
        globally_attended_tokens
    ])

    # Restore the reduced token sequence to a spatial feature map.
    attended_spatial_features = layers.Reshape(
        (pooled_height, pooled_width, channel_count),
        name=f"{name}_reshape"
    )(attention_residual_tokens)

    restored_spatial_features = layers.Resizing(
        feature_height,
        feature_width,
        interpolation="bilinear",
        name=f"{name}_restore"
    )(attended_spatial_features)

    projected_global_attention_features = layers.Conv2D(
        channel_count,
        1,
        padding="same",
        name=f"{name}_project"
    )(restored_spatial_features)

    return projected_global_attention_features


# ------------------------------------------------------------
# Local spatial attention view
# ------------------------------------------------------------
def local_attention_view(input_features,
                         name="local_attention"):
    """Local spatial attention using depthwise convolution."""

    channel_count = tf.keras.backend.int_shape(
        input_features
```

```
    )[-1]

    local_depthwise_features = layers.DepthwiseConv2D(
        3,
        padding="same",
        name=f"{name}_depthwise"
    )(input_features)

    local_attention_weights = layers.Conv2D(
        channel_count,
        1,
        padding="same",
        activation="sigmoid",
        name=f"{name}_gate"
    )(local_depthwise_features)

    locally_refined_features = layers.Multiply(
        name=f"{name}_apply"
    )([
        input_features,
        local_attention_weights
    ])

    return locally_refined_features


# ---------------------------------------------------------------
# Multi-view attention and feature fusion
# ---------------------------------------------------------------
def multi_view_attention_block(input_features, filters,
                               name="mva"):
    """
    Channel, global, and local attention views followed by fusion.
    """

    # Project the input to a common channel dimension.
    projected_input_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        name=f"{name}_input_projection"
    )(input_features)

    # Channel attention branch.
    channel_attention_features = channel_attention(
        projected_input_features,
        name=f"{name}_channel"
    )
```

```
# Global self-attention branch.
global_attention_features = global_attention_view(
    projected_input_features,
    name=f"{name}_global"
)

# Local spatial attention branch.
local_attention_features = local_attention_view(
    projected_input_features,
    name=f"{name}_local"
)

# Concatenate all attention views.
concatenated_attention_features = layers.Concatenate(
    name=f"{name}_merge"
)([
    channel_attention_features,
    global_attention_features,
    local_attention_features
])

fused_attention_features = layers.Conv2D(
    filters,
    1,
    padding="same",
    use_bias=False,
    name=f"{name}_merge_projection"
)(concatenated_attention_features)

normalized_attention_features = layers.LayerNormalization(
    name=f"{name}_merge_bn"
)(fused_attention_features)

# Residual fusion with the projected input.
residual_attention_features = layers.Add(
    name=f"{name}_residual_add"
)([
    projected_input_features,
    normalized_attention_features
])

output_features = layers.Activation(
    "relu",
    name=f"{name}_out_relu"
)(residual_attention_features)

return output_features
```

```
# ------------------------------------------------------------
# Encoder layer
# ------------------------------------------------------------
def encoder_layer(input_features, filters, name):
    """Downsampling encoder with ResCon and multi-view attention."""

    downsampled_features = conv2d_block(
        input_features,
        filters,
        kernel_size=3,
        strides=2,
        name=f"{name}_down"
    )

    residual_conformer_features = rescon_block(
        downsampled_features,
        filters,
        name=f"{name}_rescon"
    )

    attention_refined_encoder_features = multi_view_attention_block(
        residual_conformer_features,
        filters,
        name=f"{name}_mva"
    )

    return attention_refined_encoder_features


# ------------------------------------------------------------
# Decoder layer with gated encoder skip connection
# ------------------------------------------------------------
def decoder_layer(input_features, encoder_skip_features,
                  filters, name):
    """Upsampling decoder with gated skip-feature fusion."""

    upsampled_decoder_features = layers.UpSampling2D(
        size=(2, 2),
        interpolation="bilinear",
        name=f"{name}_up"
    )(input_features)

    convolved_decoder_features = conv2d_block(
        upsampled_decoder_features,
        filters,
        kernel_size=3,
        strides=1,
        name=f"{name}_up_conv"
    )
```

```
# Align the encoder skip feature-map dimensions.
decoder_height = tf.keras.backend.int_shape(
    convolved_decoder_features
)[1]

decoder_width = tf.keras.backend.int_shape(
    convolved_decoder_features
)[2]

skip_height = tf.keras.backend.int_shape(
    encoder_skip_features
)[1]

skip_width = tf.keras.backend.int_shape(
    encoder_skip_features
)[2]

if (
    skip_height != decoder_height
    or skip_width != decoder_width
):
    encoder_skip_features = layers.Resizing(
        decoder_height,
        decoder_width,
        interpolation="bilinear",
        name=f"{name}_skip_resize"
    )(encoder_skip_features)

projected_skip_features = layers.Conv2D(
    filters,
    1,
    padding="same",
    name=f"{name}_skip_projection"
)(encoder_skip_features)

# Compute the skip-connection gate.
decoder_skip_gate_input = layers.Concatenate(
    name=f"{name}_gate_concat"
)([
    convolved_decoder_features,
    projected_skip_features
])

skip_gate_logits = layers.Conv2D(
    filters,
    1,
    padding="same",
```

```
        name=f"{name}_gate_conv"
    )(decoder_skip_gate_input)

    skip_attention_weights = layers.Activation(
        "sigmoid",
        name=f"{name}_gate_sigmoid"
    )(skip_gate_logits)

    sigmoid_gated_skip_features = layers.Multiply(
        name=f"{name}_skip_sigmoid_product"
    )([
        projected_skip_features,
        skip_attention_weights
    ])

    tanh_gated_skip_features = layers.Activation(
        "tanh",
        name=f"{name}_skip_tanh"
    )(sigmoid_gated_skip_features)

    # Fuse the gated skip features with decoder features.
    fused_decoder_skip_features = layers.Add(
        name=f"{name}_skip_add"
    )([
        convolved_decoder_features,
        tanh_gated_skip_features
    ])

    activated_decoder_skip_features = layers.Activation(
        "relu",
        name=f"{name}_skip_relu"
    )(fused_decoder_skip_features)

    residual_conformer_decoder_features = rescon_block(
        activated_decoder_skip_features,
        filters,
        name=f"{name}_rescon"
    )

    attention_refined_decoder_features = multi_view_attention_block(
        residual_conformer_decoder_features,
        filters,
        name=f"{name}_mva"
    )

    return attention_refined_decoder_features


# ----------------------------------------------------------------
```

```
# Build MANNER-inspired non-causal spectrogram model
# Input/output shape: (128, 128, 1)
# ------------------------------------------------------------
def build_manner_inspired_noncausal(
    input_shape=(128, 128, 1),
    base_filters=16,
    learning_rate=1e-3
):
    noisy_spectrogram_input = layers.Input(
        shape=input_shape,
        name="noisy_spectrogram"
    )

    # ------------------------------------------------------------
    # Initial feature projection
    # ------------------------------------------------------------
    initial_spectrogram_features = conv2d_block(
        noisy_spectrogram_input,
        base_filters,
        kernel_size=3,
        strides=1,
        name="input_projection"
    )

    # ------------------------------------------------------------
    # Encoder: 128 -> 64 -> 32 -> 16 -> 8
    # ------------------------------------------------------------
    encoder1_features = encoder_layer(
        initial_spectrogram_features,
        base_filters,
        "encoder1"
    )

    encoder2_features = encoder_layer(
        encoder1_features,
        base_filters * 2,
        "encoder2"
    )

    encoder3_features = encoder_layer(
        encoder2_features,
        base_filters * 4,
        "encoder3"
    )

    encoder4_features = encoder_layer(
        encoder3_features,
        base_filters * 8,
```

```
    "encoder4"
)

# ------------------------------------------------------------
# Bottleneck: ResCon and multi-view attention
# ------------------------------------------------------------
bottleneck_residual_conformer_features = rescon_block(
    encoder4_features,
    base_filters * 8,
    name="bottleneck_rescon"
)

bottleneck_attention_features = multi_view_attention_block(
    bottleneck_residual_conformer_features,
    base_filters * 8,
    name="bottleneck_mva"
)

# ------------------------------------------------------------
# Decoder with gated encoder skip connections
# ------------------------------------------------------------
decoder3_features = decoder_layer(
    bottleneck_attention_features,
    encoder3_features,
    base_filters * 4,
    "decoder3"
)

decoder2_features = decoder_layer(
    decoder3_features,
    encoder2_features,
    base_filters * 2,
    "decoder2"
)

decoder1_features = decoder_layer(
    decoder2_features,
    encoder1_features,
    base_filters,
    "decoder1"
)

# ------------------------------------------------------------
# Final upsampling: 64x64 -> 128x128
# ------------------------------------------------------------
final_upsampled_decoder_features = layers.UpSampling2D(
    size=(2, 2),
    interpolation="bilinear",
```

```
        name="decoder_final_up"
    )(decoder1_features)

    final_decoder_features = conv2d_block(
        final_upsampled_decoder_features,
        base_filters,
        kernel_size=3,
        strides=1,
        name="decoder_final_conv"
    )

    # ------------------------------------------------------------
    # Enhanced magnitude spectrogram prediction
    # ------------------------------------------------------------
    enhanced_spectrogram_output = layers.Conv2D(
        1,
        kernel_size=1,
        padding="same",
        activation="relu",
        name="enhanced_spectrogram"
    )(final_decoder_features)

    manner_inspired_model = Model(
        inputs=noisy_spectrogram_input,
        outputs=enhanced_spectrogram_output,
        name="MANNER_Inspired_NonCausal_2D"
    )

    manner_inspired_model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=learning_rate
        ),
        loss="mae",
        metrics=["mse"]
    )

    return manner_inspired_model


# ------------------------------------------------------------
# Build model, display architecture, and test output shape
# ------------------------------------------------------------
if __name__ == "__main__":
    manner_model = build_manner_inspired_noncausal(
        input_shape=(128, 128, 1),
        base_filters=16,
        learning_rate=1e-3
    )
```

```
manner_model.summary()
```

6. TFA-S-TCN [8]: Multi-stage progressive learning-based speech enhancement using time-frequency attentive squeezed temporal convolutional networks

Non-Causal implementation code for TFA-S-TCN

```
import tensorflow as tf
from tensorflow.keras import layers, Model


def time_frequency_attention(input_features, name, reduction=8):
    """Parallel time-frame and frequency attention for [B, T, F, C]."""
    channel_count = int(input_features.shape[-1])
    reduced_channel_count = max(channel_count // reduction, 4)

    # Time-frame attention: average across frequency.
    time_frame_descriptor = layers.Lambda(
        lambda tensor: tf.reduce_mean(tensor, axis=2),
        name=f"{name}_time_average_pool"
    )(input_features)

    time_attention_features = layers.Conv1D(
        reduced_channel_count,
        3,
        padding="same",
        activation="relu",
        name=f"{name}_time_conv_relu"
    )(time_frame_descriptor)

    time_attention_weights = layers.Conv1D(
        1,
        1,
        padding="same",
        activation="sigmoid",
        name=f"{name}_time_conv_sigmoid"
    )(time_attention_features)

    time_attention_weights = layers.Lambda(
        lambda tensor: tf.expand_dims(tensor, axis=2),
        name=f"{name}_time_gate_expand"
    )(time_attention_weights)

    # Frequency attention: average across time.
    frequency_descriptor = layers.Lambda(
        lambda tensor: tf.reduce_mean(tensor, axis=1),
        name=f"{name}_frequency_average_pool"
```

```
)(input_features)

frequency_attention_features = layers.Conv1D(
    reduced_channel_count,
    3,
    padding="same",
    activation="relu",
    name=f"{name}_frequency_conv_relu"
)(frequency_descriptor)

frequency_attention_weights = layers.Conv1D(
    1,
    1,
    padding="same",
    activation="sigmoid",
    name=f"{name}_frequency_conv_sigmoid"
)(frequency_attention_features)

frequency_attention_weights = layers.Lambda(
    lambda tensor: tf.expand_dims(tensor, axis=1),
    name=f"{name}_frequency_gate_expand"
)(frequency_attention_weights)

time_attended_features = layers.Multiply(
    name=f"{name}_apply_time_gate"
)([input_features, time_attention_weights])

frequency_attended_features = layers.Multiply(
    name=f"{name}_apply_frequency_gate"
)([input_features, frequency_attention_weights])

combined_attention_features = layers.Add(
    name=f"{name}_combine_attention"
)([time_attended_features, frequency_attended_features])

projected_attention_features = layers.Conv2D(
    channel_count,
    1,
    padding="same",
    name=f"{name}_attention_projection"
)(combined_attention_features)

attention_residual_features = layers.Add(
    name=f"{name}_attention_residual"
)([input_features, projected_attention_features])

return attention_residual_features
```

```
def squeezed_tcn_block(input_features, channels, dilation, name):
    """Residual gated temporal convolution block inspired by S-TCN."""

    residual_shortcut_features = layers.Conv2D(
        channels,
        1,
        padding="same",
        name=f"{name}_shortcut"
    )(input_features)

    main_pointwise_features = layers.Conv2D(
        channels,
        1,
        padding="same",
        use_bias=False,
        name=f"{name}_main_pointwise"
    )(input_features)

    normalized_main_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_main_norm1"
    )(main_pointwise_features)

    activated_main_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_main_prelu1"
    )(normalized_main_features)

    dilated_temporal_features = layers.SeparableConv2D(
        channels,
        kernel_size=(3, 1),
        dilation_rate=(dilation, 1),
        padding="same",
        use_bias=False,
        name=f"{name}_main_dilated_temporal"
    )(activated_main_features)

    normalized_temporal_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_main_norm2"
    )(dilated_temporal_features)

    activated_temporal_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_main_prelu2"
    )(normalized_temporal_features)

    # Gating branch.
```

```
    gate_pointwise_features = layers.Conv2D(
        channels,
        1,
        padding="same",
        name=f"{name}_gate_pointwise"
    )(input_features)

    dilated_gate_features = layers.SeparableConv2D(
        channels,
        kernel_size=(3, 1),
        dilation_rate=(dilation, 1),
        padding="same",
        name=f"{name}_gate_dilated_temporal"
    )(gate_pointwise_features)

    temporal_gate_weights = layers.Activation(
        "sigmoid",
        name=f"{name}_gate_sigmoid"
    )(dilated_gate_features)

    gated_temporal_features = layers.Multiply(
        name=f"{name}_gated_product"
    )([activated_temporal_features, temporal_gate_weights])

    projected_temporal_features = layers.Conv2D(
        channels,
        1,
        padding="same",
        name=f"{name}_output_projection"
    )(gated_temporal_features)

    residual_fused_features = layers.Add(
        name=f"{name}_residual_add"
    )([residual_shortcut_features, projected_temporal_features])

    normalized_residual_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_output_norm"
    )(residual_fused_features)

    output_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_output_prelu"
    )(normalized_residual_features)

    return output_features


def feature_fusion_block(
```

```
    previous_spectrogram_estimate,
    original_input_features,
    channels,
    name
):
    """Fuse the previous stage estimate features with original input
features."""

    previous_estimate_features = layers.Conv2D(
        channels,
        1,
        padding="same",
        name=f"{name}_previous_projection"
    )(previous_spectrogram_estimate)

    activated_previous_estimate_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_previous_prelu"
    )(previous_estimate_features)

    normalized_previous_estimate_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_previous_gln"
    )(activated_previous_estimate_features)

    projected_original_features = layers.Conv2D(
        channels,
        1,
        padding="same",
        name=f"{name}_original_projection"
    )(original_input_features)

    activated_original_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_original_prelu"
    )(projected_original_features)

    normalized_original_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_original_gln"
    )(activated_original_features)

    added_fusion_features = layers.Add(
        name=f"{name}_feature_add"
    )([
        normalized_previous_estimate_features,
        normalized_original_features
    ])
```

```
    projected_fusion_features = layers.Conv2D(
        channels,
        1,
        padding="same",
        name=f"{name}_fusion_projection"
    )(added_fusion_features)

    activated_fusion_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_fusion_prelu"
    )(projected_fusion_features)

    normalized_fusion_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_fusion_gln"
    )(activated_fusion_features)

    return normalized_fusion_features


def progressive_stage(
    stage_input_features,
    channels,
    stage_number,
    blocks_per_stage=6
):
    """Estimate a spectrogram residual at one progressive enhancement
stage."""

    stage_name = f"stage{stage_number}"

    stage_projection_features = layers.Conv2D(
        channels,
        1,
        padding="same",
        name=f"{stage_name}_input_projection"
    )(stage_input_features)

    stage_attention_features = time_frequency_attention(
        stage_projection_features,
        f"{stage_name}_tfa"
    )

    temporal_context_features = stage_attention_features

    for block_index in range(blocks_per_stage):
        dilation_rate = 2 ** (block_index % 5)
```

```
            temporal_context_features = squeezed_tcn_block(
                temporal_context_features,
                channels,
                dilation_rate,
                f"{stage_name}_stcn{block_index + 1}"
            )

        predicted_spectrogram_residual = layers.Conv2D(
            1,
            1,
            padding="same",
            name=f"{stage_name}_residual_spectrogram"
        )(temporal_context_features)

        # The first input channel carries the previous stage's estimate.
        previous_stage_estimate = layers.Lambda(
            lambda tensor: tensor[..., :1],
            name=f"{stage_name}_current_estimate"
        )(stage_input_features)

        enhanced_stage_estimate = layers.Add(
            name=f"{stage_name}_progressive_estimate"
        )([
            previous_stage_estimate,
            predicted_spectrogram_residual
        ])

        return enhanced_stage_estimate


    def build_five_stage_noncausal_model(
        input_shape=(128, 128, 1),
        channels=32,
        stages=5,
        blocks_per_stage=6
    ):
        """Build the five-stage, noncausal TFA-S-TCN-inspired model."""

        if stages != 5:
            raise ValueError(
                "This requested configuration is for exactly five stages."
            )

        noisy_spectrogram_input = layers.Input(
            shape=input_shape,
            name="noisy_magnitude_spectrogram"
        )

        original_input_features = layers.Conv2D(
```

```
        channels,
        3,
        padding="same",
        activation="relu",
        name="original_feature_extractor"
    )(noisy_spectrogram_input)

    previous_stage_estimate = noisy_spectrogram_input
    progressive_stage_estimates = []

    for stage_number in range(1, stages + 1):

        if stage_number == 1:
            stage_input_features = layers.Concatenate(
                name="stage1_input_fusion"
            )([
                previous_stage_estimate,
                original_input_features
            ])

        else:
            fused_original_and_estimate_features =
feature_fusion_block(
                previous_stage_estimate,
                original_input_features,
                channels,
                f"stage{stage_number}_fb"
            )

            stage_input_features = layers.Concatenate(
                name=f"stage{stage_number}_input_fusion"
            )([
                previous_stage_estimate,
                fused_original_and_estimate_features
            ])

        previous_stage_estimate = progressive_stage(
            stage_input_features,
            channels,
            stage_number,
            blocks_per_stage=blocks_per_stage
        )

        progressive_stage_estimates.append(previous_stage_estimate)

    five_stage_model = Model(
        inputs=noisy_spectrogram_input,
        outputs=progressive_stage_estimates,
```

```
        name="FiveStage_NonCausal_TFA_STCN"
    )

    return five_stage_model


def spectral_loss(y_true, y_pred):
    """Magnitude-spectrogram loss; not a waveform reconstruction
loss."""

    mean_absolute_spectral_error = tf.reduce_mean(
        tf.abs(y_true - y_pred)
    )

    mean_squared_spectral_error = tf.reduce_mean(
        tf.square(y_true - y_pred)
    )

    combined_spectral_loss = (
        mean_absolute_spectral_error
        + 0.5 * mean_squared_spectral_error
    )

    return combined_spectral_loss


if __name__ == "__main__":

    model = build_five_stage_noncausal_model(
        input_shape=(128, 128, 1),
        channels=32,
        stages=5,
        blocks_per_stage=6
    )

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss=[spectral_loss] * 5
    )

    model.summary()
```

7. HATFANet [15]: HYBRID ATTENTION TIME-FREQUENCY ANALYSIS NETWORK FOR SINGLE-CHANNEL SPEECH ENHANCEMENT (HATFANet)

Non-Causal implementation code for HATFANet

```
import tensorflow as tf
from tensorflow.keras import layers, Model


# HATFANet-inspired noncausal 2D spectrogram adaptation.
# This is not the official author implementation or an exact
reproduction
# of the original waveform-domain paper.


def conv_bn_act(
    input_features,
    filters,
    kernel_size=3,
    strides=1,
    activation="gelu",
    name="conv"
):
    """Convolution followed by layer normalization and optional
activation."""

    convolution_features = layers.Conv2D(
        filters,
        kernel_size,
        strides=strides,
        padding="same",
        use_bias=False,
        name=f"{name}_conv"
    )(input_features)

    normalized_features = layers.LayerNormalization(
        name=f"{name}_bn"
    )(convolution_features)

    if activation:
        activated_features = layers.Activation(
            activation,
            name=f"{name}_{activation}"
        )(normalized_features)
        return activated_features

    return normalized_features


def match_spatial(input_features, reference_features, name):
    """Resize feature maps to match the reference spatial
dimensions."""

    reference_shape = tf.keras.backend.int_shape(reference_features)
```

```
    if reference_shape[1] is None or reference_shape[2] is None:
        raise ValueError("Fixed spatial dimensions are required.")

    input_shape = tf.keras.backend.int_shape(input_features)

    if input_shape[1:3] != reference_shape[1:3]:
        resized_features = layers.Resizing(
            reference_shape[1],
            reference_shape[2],
            interpolation="bilinear",
            name=name
        )(input_features)
        return resized_features

    return input_features


def wcab(input_features, name="wcab", reduction=4):
    """Spatial and channel attention block."""

    channel_count = int(
        tf.keras.backend.int_shape(input_features)[-1]
    )
    reduced_channel_count = max(channel_count // reduction, 4)

    # Spatial-attention branch.
    spatial_attention_features = layers.Conv2D(
        reduced_channel_count,
        3,
        padding="same",
        activation="gelu",
        name=f"{name}_spatial_conv"
    )(input_features)

    spatial_attention_weights = layers.Conv2D(
        1,
        1,
        padding="same",
        activation="sigmoid",
        name=f"{name}_spatial_gate"
    )(spatial_attention_features)

    spatially_attended_features = layers.Multiply(
        name=f"{name}_spatial_apply"
    )([input_features, spatial_attention_weights])

    # Channel-attention branch.
    global_channel_descriptor = layers.GlobalAveragePooling2D(
```

```
        keepdims=True,
        name=f"{name}_gap"
    )(input_features)

    reduced_channel_features = layers.Conv2D(
        reduced_channel_count,
        1,
        activation="relu",
        name=f"{name}_channel_reduce"
    )(global_channel_descriptor)

    channel_attention_weights = layers.Conv2D(
        channel_count,
        1,
        activation="sigmoid",
        name=f"{name}_channel_expand"
    )(reduced_channel_features)

    channel_attended_features = layers.Multiply(
        name=f"{name}_channel_apply"
    )([input_features, channel_attention_weights])

    # Combine attention branches and apply residual refinement.
    combined_attention_features = layers.Add(
        name=f"{name}_add"
    )([
        spatially_attended_features,
        channel_attended_features
    ])

    projected_attention_features = layers.Conv2D(
        channel_count,
        1,
        padding="same",
        name=f"{name}_project"
    )(combined_attention_features)

    residual_attention_features = layers.Add(
        name=f"{name}_residual"
    )([
        input_features,
        projected_attention_features
    ])

    return residual_attention_features


def hab(input_features, filters, name="hab"):
    """Multi-view hierarchical attention block."""
```

```
projected_input_features = conv_bn_act(
    input_features,
    filters,
    kernel_size=1,
    name=f"{name}_projection"
)

spatial_height, spatial_width = (
    tf.keras.backend.int_shape(projected_input_features)[1:3]
)

attention_view_features = []

# Block, grid, and overlap-inspired spatial views.
for view_name, partition_divisor in (
    ("block", 2),
    ("grid", 4),
    ("overlap", 2)
):
    partition_height = max(
        1, spatial_height // partition_divisor
    )
    partition_width = max(
        1, spatial_width // partition_divisor
    )

    partitioned_features = layers.Resizing(
        partition_height,
        partition_width,
        interpolation="bilinear",
        name=f"{name}_{view_name}_partition"
    )(projected_input_features)

    normalized_partition_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_{view_name}_norm"
    )(partitioned_features)

    expanded_partition_features = layers.Conv2D(
        filters * 2,
        1,
        padding="same",
        name=f"{name}_{view_name}_mlp_expand"
    )(normalized_partition_features)

    value_features, gate_features = layers.Lambda(
        lambda tensor: tf.split(tensor, 2, axis=-1),
```

```
            name=f"{name}_{view_name}_split"
        )(expanded_partition_features)

        sigmoid_gate_weights = layers.Activation(
            "sigmoid",
            name=f"{name}_{view_name}_gate"
        )(gate_features)

        gated_partition_features = layers.Multiply(
            name=f"{name}_{view_name}_gated"
        )([value_features, sigmoid_gate_weights])

        projected_view_features = layers.Conv2D(
            filters,
            1,
            padding="same",
            name=f"{name}_{view_name}_mlp_project"
        )(gated_partition_features)

        restored_view_features = layers.Resizing(
            spatial_height,
            spatial_width,
            interpolation="bilinear",
            name=f"{name}_{view_name}_unpartition"
        )(projected_view_features)

        attention_refined_view_features = wcab(
            restored_view_features,
            name=f"{name}_{view_name}_wcab"
        )

        attention_view_features.append(attention_refined_view_features)

    concatenated_attention_views = layers.Concatenate(
        name=f"{name}_views_concat"
    )(attention_view_features)

    fused_attention_features = conv_bn_act(
        concatenated_attention_views,
        filters,
        kernel_size=1,
        activation=None,
        name=f"{name}_fusion"
    )

    residual_attention_features = layers.Add(
        name=f"{name}_residual"
    )([
```

```
            projected_input_features,
            fused_attention_features
        ])

        normalized_output_features = layers.LayerNormalization(
            axis=-1,
            name=f"{name}_output_norm"
        )(residual_attention_features)

        return normalized_output_features


    def encoder_layer(input_features, filters, name):
        """Encoder downsampling followed by hierarchical attention."""

        downsampled_encoder_features = conv_bn_act(
            input_features,
            filters,
            strides=2,
            name=f"{name}_down"
        )

        attention_refined_encoder_features = hab(
            downsampled_encoder_features,
            filters,
            name=f"{name}_hab"
        )

        return attention_refined_encoder_features


    def decoder_layer(
        decoder_input_features,
        skip_connection_features,
        filters,
        name
    ):
        """Decoder upsampling, skip fusion, and hierarchical attention."""

        upsampled_decoder_features = layers.UpSampling2D(
            (2, 2),
            interpolation="bilinear",
            name=f"{name}_upsample"
        )(decoder_input_features)

        convolved_decoder_features = conv_bn_act(
            upsampled_decoder_features,
            filters,
            name=f"{name}_conv"
```

```
    )

    resized_skip_features = match_spatial(
        skip_connection_features,
        convolved_decoder_features,
        f"{name}_skip_resize"
    )

    projected_skip_features = conv_bn_act(
        resized_skip_features,
        filters,
        kernel_size=1,
        activation=None,
        name=f"{name}_skip_projection"
    )

    concatenated_decoder_skip_features = layers.Concatenate(
        name=f"{name}_concat"
    )([
        convolved_decoder_features,
        projected_skip_features
    ])

    fused_decoder_features = conv_bn_act(
        concatenated_decoder_skip_features,
        filters,
        name=f"{name}_fusion"
    )

    attention_refined_decoder_features = hab(
        fused_decoder_features,
        filters,
        name=f"{name}_hab"
    )

    return attention_refined_decoder_features


def build_hatfanet_inspired_noncausal(
    input_shape=(128, 128, 1),
    base_filters=16,
    learning_rate=1e-3
):
    """Build a HATFANet-inspired spectrogram enhancement network."""

    noisy_spectrogram_input = layers.Input(
        shape=input_shape,
        name="noisy_spectrogram"
    )
```

```
# Initial feature extraction.
encoder_stem_features = conv_bn_act(
    noisy_spectrogram_input,
    base_filters,
    name="dense_encoder_stem"
)

# Four-level encoder.
encoder1_features = encoder_layer(
    encoder_stem_features,
    base_filters,
    "encoder1"
)  # 64 x 64

encoder2_features = encoder_layer(
    encoder1_features,
    base_filters * 2,
    "encoder2"
)  # 32 x 32

encoder3_features = encoder_layer(
    encoder2_features,
    base_filters * 4,
    "encoder3"
)  # 16 x 16

encoder4_features = encoder_layer(
    encoder3_features,
    base_filters * 8,
    "encoder4"
)  # 8 x 8

# Multi-level embedding fusion at bottleneck resolution.
encoder1_bottleneck_features = layers.Resizing(
    8, 8,
    interpolation="bilinear",
    name="fusion_resize_e1"
)(encoder1_features)

encoder2_bottleneck_features = layers.Resizing(
    8, 8,
    interpolation="bilinear",
    name="fusion_resize_e2"
)(encoder2_features)

encoder3_bottleneck_features = layers.Resizing(
    8, 8,
```

```
    interpolation="bilinear",
    name="fusion_resize_e3"
)(encoder3_features)

concatenated_multilevel_features = layers.Concatenate(
    name="embedding_fusion_concat"
)([
    encoder1_bottleneck_features,
    encoder2_bottleneck_features,
    encoder3_bottleneck_features,
    encoder4_features
])

projected_bottleneck_features = conv_bn_act(
    concatenated_multilevel_features,
    base_filters * 8,
    kernel_size=1,
    name="embedding_fusion_projection"
)

bottleneck_attention_features = hab(
    projected_bottleneck_features,
    base_filters * 8,
    name="embedding_fusion_hab"
)

# Three decoder stages.
decoder3_features = decoder_layer(
    bottleneck_attention_features,
    encoder3_features,
    base_filters * 4,
    "decoder3"
)  # 16 x 16

decoder2_features = decoder_layer(
    decoder3_features,
    encoder2_features,
    base_filters * 2,
    "decoder2"
)  # 32 x 32

decoder1_features = decoder_layer(
    decoder2_features,
    encoder1_features,
    base_filters,
    "decoder1"
)  # 64 x 64
```

```
    final_upsampled_decoder_features = layers.UpSampling2D(
        (2, 2),
        interpolation="bilinear",
        name="decoder_final_upsample"
    )(decoder1_features)

    final_decoder_features = conv_bn_act(
        final_upsampled_decoder_features,
        base_filters,
        name="decoder_final_conv"
    )

    # Estimate a magnitude mask and apply it to the noisy input.
    estimated_magnitude_mask = layers.Conv2D(
        1,
        1,
        padding="same",
        activation="sigmoid",
        name="estimated_magnitude_mask"
    )(final_decoder_features)

    enhanced_magnitude_spectrogram = layers.Multiply(
        name="enhanced_spectrogram"
    )([
        noisy_spectrogram_input,
        estimated_magnitude_mask
    ])

    hatfanet_inspired_model = Model(
        inputs=noisy_spectrogram_input,
        outputs=enhanced_magnitude_spectrogram,
        name="HATFANet_Inspired_NonCausal_2D"
    )

    hatfanet_inspired_model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=learning_rate
        ),
        loss="mae",
        metrics=["mse"]
    )

    return hatfanet_inspired_model


if __name__ == "__main__":

    model = build_hatfanet_inspired_noncausal(
        input_shape=(128, 128, 1),
```

```
        base_filters=16,
        learning_rate=1e-3
    )

    model.summary()
```

8. ROSE-Net [16]: A Recognition-Oriented Speech Enhancement Framework in Air Traffic Control Using Multi-Objective Learning

Non-Causal implementation code for ROSE-Net

```
import tensorflow as tf
from tensorflow.keras import layers, Model


def conv_block(input_features, filters, name):
    """Two convolutional layers with normalization and ReLU
activation."""

    first_convolution_features = layers.Conv2D(
        filters,
        3,
        padding="same",
        use_bias=False,
        name=f"{name}_conv1"
    )(input_features)

    first_normalized_features = layers.LayerNormalization(
        name=f"{name}_bn1"
    )(first_convolution_features)

    first_activated_features = layers.ReLU(
        name=f"{name}_relu1"
    )(first_normalized_features)

    second_convolution_features = layers.Conv2D(
        filters,
        3,
        padding="same",
        use_bias=False,
        name=f"{name}_conv2"
    )(first_activated_features)

    second_normalized_features = layers.LayerNormalization(
        name=f"{name}_bn2"
    )(second_convolution_features)

    output_features = layers.ReLU(
```

```
        name=f"{name}_relu2"
    )(second_normalized_features)

    return output_features


def csatt(input_features, reduction=8, name="csatt"):
    """Channel and spatial-sequence attention (ROSE-inspired)."""

    channel_count = int(input_features.shape[-1])
    reduced_channel_count = max(channel_count // reduction, 4)

    # Channel-attention path.
    global_channel_descriptor = layers.GlobalAveragePooling2D(
        keepdims=True,
        name=f"{name}_gap"
    )(input_features)

    reduced_channel_features = layers.Conv2D(
        reduced_channel_count,
        1,
        activation="relu",
        padding="same",
        name=f"{name}_channel_reduce"
    )(global_channel_descriptor)

    channel_attention_weights = layers.Conv2D(
        channel_count,
        1,
        activation="sigmoid",
        padding="same",
        name=f"{name}_channel_gate"
    )(reduced_channel_features)

    channel_attended_features = layers.Multiply(
        name=f"{name}_channel_apply"
    )([
        input_features,
        channel_attention_weights
    ])

    # Spatial-sequence attention path.
    sequence_reduced_features = layers.Conv2D(
        1,
        1,
        padding="same",
        activation="relu",
        name=f"{name}_sequence_reduce"
    )(input_features)
```

```
    sequence_attention_weights = layers.Conv2D(
        1,
        7,
        padding="same",
        activation="sigmoid",
        name=f"{name}_sequence_gate"
    )(sequence_reduced_features)

    sequence_attended_features = layers.Multiply(
        name=f"{name}_sequence_apply"
    )([
        input_features,
        sequence_attention_weights
    ])

    # Merge the channel and sequence attention paths.
    dual_path_attention_features = layers.Add(
        name=f"{name}_dual_path_add"
    )([
        channel_attended_features,
        sequence_attended_features
    ])

    projected_attention_features = layers.Conv2D(
        channel_count,
        1,
        padding="same",
        use_bias=False,
        name=f"{name}_project"
    )(dual_path_attention_features)

    normalized_attention_features = layers.LayerNormalization(
        name=f"{name}_project_bn"
    )(projected_attention_features)

    residual_attention_features = layers.Add(
        name=f"{name}_residual_add"
    )([
        input_features,
        normalized_attention_features
    ])

    return residual_attention_features


def absf(decoder_features, encoder_skip_features, filters,
name="absf"):
    """Attention-based skip fusion."""
```

```
projected_decoder_features = layers.Conv2D(
    filters,
    1,
    padding="same",
    name=f"{name}_decoder_proj"
)(decoder_features)

projected_encoder_features = layers.Conv2D(
    filters,
    1,
    padding="same",
    name=f"{name}_encoder_proj"
)(encoder_skip_features)

combined_attention_score_features = layers.Add(
    name=f"{name}_score_add"
)([
    projected_decoder_features,
    projected_encoder_features
])

activated_attention_score_features = layers.ReLU(
    name=f"{name}_score_relu"
)(combined_attention_score_features)

spatial_skip_attention_mask = layers.Conv2D(
    1,
    1,
    padding="same",
    activation="sigmoid",
    name=f"{name}_mask"
)(activated_attention_score_features)

gated_encoder_skip_features = layers.Multiply(
    name=f"{name}_gated_skip"
)([
    encoder_skip_features,
    spatial_skip_attention_mask
])

projected_gated_skip_features = layers.Conv2D(
    filters,
    1,
    padding="same",
    name=f"{name}_gated_skip_proj"
)(gated_encoder_skip_features)
```

```
    fused_decoder_skip_features = layers.Add(
        name=f"{name}_fused_add"
    )([
        projected_gated_skip_features,
        projected_decoder_features
    ])

    return fused_decoder_skip_features


def encoder_stage(input_features, filters, name):
    """Encoder feature extraction, attention refinement, and
downsampling."""

    extracted_encoder_features = conv_block(
        input_features,
        filters,
        f"{name}_features"
    )

    attention_refined_encoder_features = csatt(
        extracted_encoder_features,
        name=f"{name}_csatt"
    )

    encoder_skip_features = attention_refined_encoder_features

    downsampled_encoder_features = layers.Conv2D(
        filters,
        3,
        strides=2,
        padding="same",
        activation="relu",
        name=f"{name}_downsample"
    )(attention_refined_encoder_features)

    return encoder_skip_features, downsampled_encoder_features


def decoder_stage(
    decoder_input_features,
    encoder_skip_features,
    filters,
    name
):
    """Decoder upsampling, attention-guided skip fusion, and
refinement."""

    upsampled_decoder_features = layers.UpSampling2D(
```

```
        (2, 2),
        interpolation="bilinear",
        name=f"{name}_upsample"
    )(decoder_input_features)

    projected_decoder_features = layers.Conv2D(
        filters,
        3,
        padding="same",
        activation="relu",
        name=f"{name}_up_project"
    )(upsampled_decoder_features)

    attention_fused_skip_features = absf(
        projected_decoder_features,
        encoder_skip_features,
        filters,
        name=f"{name}_absf"
    )

    refined_decoder_features = conv_block(
        attention_fused_skip_features,
        filters,
        f"{name}_features"
    )

    attention_refined_decoder_features = csatt(
        refined_decoder_features,
        name=f"{name}_csatt"
    )

    return attention_refined_decoder_features


def build_rose_inspired(input_shape=(128, 128, 1), base_filters=16):
    """Build a ROSE-inspired noncausal spectrogram enhancement
network."""

    noisy_spectrogram_input = layers.Input(
        shape=input_shape,
        name="noisy_spectrogram"
    )

    # Encoder stages and their skip connections.
    encoder1_skip_features, encoder1_downsampled_features =
encoder_stage(
        noisy_spectrogram_input,
        base_filters,
        "encoder1"
```

```
    )  # Skip: 128 x 128

    encoder2_skip_features, encoder2_downsampled_features =
encoder_stage(
        encoder1_downsampled_features,
        base_filters * 2,
        "encoder2"
    )  # Skip: 64 x 64

    encoder3_skip_features, encoder3_downsampled_features =
encoder_stage(
        encoder2_downsampled_features,
        base_filters * 4,
        "encoder3"
    )  # Skip: 32 x 32

    encoder4_skip_features, encoder4_downsampled_features =
encoder_stage(
        encoder3_downsampled_features,
        base_filters * 8,
        "encoder4"
    )  # Skip: 16 x 16

    # Bottleneck feature extraction and attention.
    bottleneck_convolution_features = conv_block(
        encoder4_downsampled_features,
        base_filters * 16,
        "bottleneck"
    )  # 8 x 8

    bottleneck_attention_features = csatt(
        bottleneck_convolution_features,
        name="bottleneck_csatt"
    )

    # Decoder stages with attention-guided skip connections.
    decoder4_features = decoder_stage(
        bottleneck_attention_features,
        encoder4_skip_features,
        base_filters * 8,
        "decoder4"
    )  # 16 x 16

    decoder3_features = decoder_stage(
        decoder4_features,
        encoder3_skip_features,
        base_filters * 4,
        "decoder3"
```

```
    )  # 32 x 32

    decoder2_features = decoder_stage(
        decoder3_features,
        encoder2_skip_features,
        base_filters * 2,
        "decoder2"
    )  # 64 x 64

    decoder1_features = decoder_stage(
        decoder2_features,
        encoder1_skip_features,
        base_filters,
        "decoder1"
    )  # 128 x 128

    # Predict the nonnegative enhanced magnitude spectrogram.
    enhanced_magnitude_spectrogram = layers.Conv2D(
        1,
        1,
        padding="same",
        activation="softplus",
        name="enhanced_magnitude_spectrogram"
    )(decoder1_features)

    rose_inspired_model = Model(
        inputs=noisy_spectrogram_input,
        outputs=enhanced_magnitude_spectrogram,
        name="ROSE_Inspired_NonCausal_TF"
    )

    return rose_inspired_model


def spectral_l1_mse_loss(y_true, y_pred):
    """Combined L1 and mean-squared error for spectrogram
prediction."""

    mean_absolute_spectral_error = tf.reduce_mean(
        tf.abs(y_true - y_pred)
    )

    mean_squared_spectral_error = tf.reduce_mean(
        tf.square(y_true - y_pred)
    )

    combined_spectral_loss = (
        mean_absolute_spectral_error
        + 0.5 * mean_squared_spectral_error
```

```
    )

    return combined_spectral_loss


if __name__ == "__main__":

    model = build_rose_inspired(
        input_shape=(128, 128, 1),
        base_filters=16
    )

    model.compile(
        optimizer=tf.keras.optimizers.Adam(1e-3),
        loss=spectral_l1_mse_loss,
        metrics=[
            tf.keras.metrics.MeanAbsoluteError(name="mae")
        ]
    )

    model.summary()
```

9. CNU-Net [17]: Causal Speech Enhancement Based on a Two-Branch Nested U-Net Architecture Using Self-Supervised Speech Embeddings

Non-Causal implementation code for CNU-Net

```
import tensorflow as tf
from tensorflow.keras import layers, Model


def cfeu_block(input_features, filters, name, depth=3):
    """CFEU-inspired dense feature extraction with residual fusion."""

    residual_projection_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        name=f"{name}_res_proj"
    )(input_features)

    dense_feature_collection = []

    input_projection_features = layers.Conv2D(
        filters,
        1,
        padding="same",
        use_bias=False,
```

```
        name=f"{name}_input_proj"
    )(input_features)

    normalized_input_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_input_ln"
    )(input_projection_features)

    activated_input_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_input_prelu"
    )(normalized_input_features)

    dense_feature_collection.append(activated_input_features)

    for layer_index in range(depth):
        # Dense connectivity: combine the block input and all prior
features.
        concatenated_dense_input_features = layers.Concatenate(
            name=f"{name}_dense_concat_{layer_index}"
        )([input_features] + dense_feature_collection)

        dense_convolution_features = layers.Conv2D(
            filters,
            3,
            padding="same",
            use_bias=False,
            name=f"{name}_conv_{layer_index}"
        )(concatenated_dense_input_features)

        normalized_dense_features = layers.LayerNormalization(
            axis=-1,
            name=f"{name}_ln_{layer_index}"
        )(dense_convolution_features)

        activated_dense_features = layers.PReLU(
            shared_axes=[1, 2],
            name=f"{name}_prelu_{layer_index}"
        )(normalized_dense_features)

        dense_feature_collection.append(activated_dense_features)

    concatenated_block_features = layers.Concatenate(
        name=f"{name}_all_features"
    )(dense_feature_collection)

    fused_dense_features = layers.Conv2D(
        filters,
```

```
            1,
            padding="same",
            name=f"{name}_fuse"
        )(concatenated_block_features)

        residual_fused_features = layers.Add(
            name=f"{name}_residual_add"
        )([
            residual_projection_features,
            fused_dense_features
        ])

        return residual_fused_features


    def cctfa_attention(input_features, name, reduction=8):
        """Channel and time-frequency attention with residual fusion."""

        channel_count = int(input_features.shape[-1])
        reduced_channel_count = max(channel_count // reduction, 4)

        # Channel-attention pathway.
        global_channel_descriptor = layers.GlobalAveragePooling2D(
            keepdims=True,
            name=f"{name}_gap"
        )(input_features)

        reduced_channel_features = layers.Conv2D(
            reduced_channel_count,
            1,
            activation="relu",
            padding="same",
            name=f"{name}_ch_reduce"
        )(global_channel_descriptor)

        channel_attention_weights = layers.Conv2D(
            channel_count,
            1,
            activation="sigmoid",
            padding="same",
            name=f"{name}_ch_gate"
        )(reduced_channel_features)

        channel_attended_features = layers.Multiply(
            name=f"{name}_ch_apply"
        )([
            input_features,
            channel_attention_weights
        ])
```

```
# Two-dimensional time-frequency attention pathway.
time_frequency_attention_features = layers.Conv2D(
    max(channel_count // 2, 4),
    3,
    padding="same",
    activation="relu",
    name=f"{name}_tf_conv"
)(input_features)

time_frequency_attention_weights = layers.Conv2D(
    1,
    1,
    padding="same",
    activation="sigmoid",
    name=f"{name}_tf_gate"
)(time_frequency_attention_features)

time_frequency_attended_features = layers.Multiply(
    name=f"{name}_tf_apply"
)([
    input_features,
    time_frequency_attention_weights
])

# Merge both attention pathways.
combined_attention_features = layers.Add(
    name=f"{name}_att_add"
)([
    channel_attended_features,
    time_frequency_attended_features
])

projected_attention_features = layers.Conv2D(
    channel_count,
    1,
    padding="same",
    name=f"{name}_proj"
)(combined_attention_features)

residual_attention_features = layers.Add(
    name=f"{name}_residual"
)([
    input_features,
    projected_attention_features
])

return residual_attention_features
```

```
def down_stage(input_features, filters, name):
    """Encoder stage with CFEU extraction, attention, and
downsampling."""

    encoder_input_convolution_features = layers.Conv2D(
        filters,
        3,
        padding="same",
        use_bias=False,
        name=f"{name}_inconv"
    )(input_features)

    normalized_encoder_input_features = layers.LayerNormalization(
        axis=-1,
        name=f"{name}_inconv_ln"
    )(encoder_input_convolution_features)

    activated_encoder_input_features = layers.PReLU(
        shared_axes=[1, 2],
        name=f"{name}_inconv_prelu"
    )(normalized_encoder_input_features)

    dense_encoder_features = cfeu_block(
        activated_encoder_input_features,
        filters,
        f"{name}_cfeu"
    )

    attention_refined_skip_features = cctfa_attention(
        dense_encoder_features,
        f"{name}_cctfa"
    )

    downsampled_encoder_features = layers.Conv2D(
        filters * 2,
        3,
        strides=2,
        padding="same",
        activation="relu",
        name=f"{name}_downsample"
    )(attention_refined_skip_features)

    return attention_refined_skip_features,
downsampled_encoder_features


def up_stage(
```

```
    decoder_input_features,
    encoder_skip_features,
    filters,
    name
):
    """Decoder upsampling, skip fusion, dense refinement, and
attention."""

    upsampled_decoder_features = layers.UpSampling2D(
        size=(2, 2),
        interpolation="bilinear",
        name=f"{name}_upsample"
    )(decoder_input_features)

    projected_decoder_features = layers.Conv2D(
        filters,
        3,
        padding="same",
        activation="relu",
        name=f"{name}_upconv"
    )(upsampled_decoder_features)

    # Nested U-Net-style skip fusion.
    concatenated_skip_features = layers.Concatenate(
        name=f"{name}_skip_concat"
    )([
        projected_decoder_features,
        encoder_skip_features
    ])

    dense_decoder_features = cfeu_block(
        concatenated_skip_features,
        filters,
        f"{name}_cfeu"
    )

    attention_refined_decoder_features = cctfa_attention(
        dense_decoder_features,
        f"{name}_cctfa"
    )

    return attention_refined_decoder_features


def build_noncausal_two_branch_unet(
    input_shape=(128, 128, 1),
    base_filters=16
):
```

```
    """Build the two-branch, noncausal spectrogram enhancement
network."""

    noisy_magnitude_spectrogram = layers.Input(
        shape=input_shape,
        name="noisy_magnitude_spectrogram"
    )

    # Encoder: preserve multiscale skip features.
    encoder1_skip_features, encoder1_downsampled_features = down_stage(
        noisy_magnitude_spectrogram,
        base_filters,
        "encoder1"
    )

    encoder2_skip_features, encoder2_downsampled_features = down_stage(
        encoder1_downsampled_features,
        base_filters * 2,
        "encoder2"
    )

    encoder3_skip_features, encoder3_downsampled_features = down_stage(
        encoder2_downsampled_features,
        base_filters * 4,
        "encoder3"
    )

    encoder4_skip_features, encoder4_downsampled_features = down_stage(
        encoder3_downsampled_features,
        base_filters * 8,
        "encoder4"
    )

    # Latent representation.
    latent_dense_features = cfeu_block(
        encoder4_downsampled_features,
        base_filters * 16,
        "latent_cfeu"
    )

    latent_attention_features = cctfa_attention(
        latent_dense_features,
        "latent_attention"
    )

    # Branch M: magnitude-mask estimation.
    mask_decoder4_features = up_stage(
        latent_attention_features,
```

```
    encoder4_skip_features,
    base_filters * 8,
    "mask_decoder4"
)

mask_decoder3_features = up_stage(
    mask_decoder4_features,
    encoder3_skip_features,
    base_filters * 4,
    "mask_decoder3"
)

mask_decoder2_features = up_stage(
    mask_decoder3_features,
    encoder2_skip_features,
    base_filters * 2,
    "mask_decoder2"
)

mask_decoder1_features = up_stage(
    mask_decoder2_features,
    encoder1_skip_features,
    base_filters,
    "mask_decoder1"
)

estimated_magnitude_mask = layers.Conv2D(
    1,
    1,
    padding="same",
    activation="sigmoid",
    name="magnitude_mask"
)(mask_decoder1_features)

# Branch S: direct clean-magnitude estimation.
spectrum_decoder4_features = up_stage(
    latent_attention_features,
    encoder4_skip_features,
    base_filters * 8,
    "spectrum_decoder4"
)

spectrum_decoder3_features = up_stage(
    spectrum_decoder4_features,
    encoder3_skip_features,
    base_filters * 4,
    "spectrum_decoder3"
)
```

```
    spectrum_decoder2_features = up_stage(
        spectrum_decoder3_features,
        encoder2_skip_features,
        base_filters * 2,
        "spectrum_decoder2"
    )

    spectrum_decoder1_features = up_stage(
        spectrum_decoder2_features,
        encoder1_skip_features,
        base_filters,
        "spectrum_decoder1"
    )

    direct_clean_magnitude_estimate = layers.Conv2D(
        1,
        1,
        padding="same",
        activation="softplus",
        name="direct_clean_magnitude"
    )(spectrum_decoder1_features)

    # Combine masked noisy magnitude and direct clean-magnitude
estimate.
    masked_noisy_magnitude_estimate = layers.Multiply(
        name="masked_noisy_magnitude"
    )([
        noisy_magnitude_spectrogram,
        estimated_magnitude_mask
    ])

    combined_magnitude_estimate = layers.Add(
        name="enhanced_magnitude"
    )([
        masked_noisy_magnitude_estimate,
        direct_clean_magnitude_estimate
    ])

    nonnegative_enhanced_magnitude = layers.Activation(
        "softplus",
        name="nonnegative_enhanced_magnitude"
    )(combined_magnitude_estimate)

    two_branch_enhancement_model = Model(
        inputs=noisy_magnitude_spectrogram,
        outputs={
            "enhanced_magnitude": nonnegative_enhanced_magnitude,
```

```
            "magnitude_mask": estimated_magnitude_mask,
            "direct_magnitude": direct_clean_magnitude_estimate,
        },
        name="NonCausal_TwoBranch_NestedUNet_Inspired"
    )

    return two_branch_enhancement_model


def enhancement_loss(y_true, y_pred):
    """Combined L1 and MSE magnitude-spectrogram loss."""

    mean_absolute_spectral_error = tf.reduce_mean(
        tf.abs(y_true - y_pred)
    )

    mean_squared_spectral_error = tf.reduce_mean(
        tf.square(y_true - y_pred)
    )

    combined_enhancement_loss = (
        mean_absolute_spectral_error
        + 0.5 * mean_squared_spectral_error
    )

    return combined_enhancement_loss


if __name__ == "__main__":

    model = build_noncausal_two_branch_unet(
        input_shape=(128, 128, 1),
        base_filters=16
    )

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=1e-3
        ),
        loss={
            "enhanced_magnitude": enhancement_loss,
            "magnitude_mask": "mse",
            "direct_magnitude": "mae",
        },
        loss_weights={
            "enhanced_magnitude": 1.0,
            "magnitude_mask": 0.1,
            "direct_magnitude": 0.1,
        },
```

```
)

model.summary()
```

supplementary material.ipynb - Colab

The proposed TF-Mamba-DPH-MSFENet was evaluated against nine baseline models: DCCRN [2], MCGN [1], MASENet [12], SADNUNet [13], MANNER [14], TFA-S-TCN [8], HATFANet [15], ROSE-Net [16], and CNUNet [17]. These baselines were independently implemented in TensorFlow using the architectural and methodological descriptions in their respective publications, as usable official implementations were not consistently available. Their implementation codes are provided in the Supplementary Material. All models were trained for up to 100 epochs with the Adam optimizer [41], an initial learning rate of 0.001, and a batch size of 16. The learning rate was halved when the validation loss failed to improve for five consecutive epochs. Where applicable, the baselines were adapted for non-causal processing through bidirectional recurrent layers, symmetric temporal convolutions, or attention without causal masking, allowing access to past and future temporal context. The corresponding modifications are summarized in Table 1. Model-specific loss functions were retained where appropriate, and their differences were considered when interpreting the comparative results.

Table 1. Implementation details of non-causal baseline models

Baseline model with Reference	**Non-causal implementation modifications**
DCCRN [2]	The bottleneck's unidirectional recurrent processing was replaced with a bidirectional LSTM (BiLSTM), enabling the recurrent component to use both forward and backward sequence context.
MCGN [1]	A bidirectional GRU (BiGRU) and symmetric `padding="same"` convolutions were used to enable the use of past and future temporal context.
MASENet [12]	Symmetric `padding="same"` temporal convolutions and unmasked attention over the complete time-frequency feature map were used to enable full-context processing.
SADNUNet [13]	Symmetric `padding="same"` convolutions and spatial self-attention were used to allow information exchange across past and future time frames.
MANNER [14]	Unmasked global self-attention and symmetric `padding="same"` convolutions were used to allow the model to exploit past and future temporal context.
TFA-S-TCN [8]	Symmetric `padding="same"` temporal convolutions and a time-attention branch aggregating information across the full sequence were used to allow future-frame information to influence the enhanced output.

HATFANet [15]	The Hybrid Attention Block (HAB) and Window and Channel Attention Block (WCAB) were implemented with unmasked attention, allowing access to both past and future time frames.
ROSE-Net [16]	Symmetric `padding="same"` convolutions and unmasked attention were used to allow the model to exploit past and future time-frame information.
CNUNet [17]	Symmetric `padding="same"` convolutions and unmasked time-frequency attention were used to enable full-context processing across time frames.